

Pair B - Road Network + Factory

Adapted from Yakov's Codebase Reading and Defence Guide (Sept 2026), split per pair by Henry. Full PDF: [docs/COMP2000_Traffic_Simulation_Codebase_Guide.pdf](#) .

You are: Callum, Jubril. **You own:** the world container - roads, lanes, bus stops - and the loader that builds it from `grid.txt` .

Files you own

```
src/trafficsim/
  factory/
    NetworkLoader.java      <- Factory - reads grid.txt into a RoadNetwork
  model/road/
    RoadNetwork.java        <- world container
    Road.java               <- one line segment
    Lane.java               <- vehicle container, back-ref to Road
    BusStop.java            <- named stop on one exact lane
  networks/
    grid.txt                <- the map data
```

Object relationships

```
RoadNetwork
  contains many Road objects
  contains many SignalisedIntersection objects  (Pair D owns these)
  contains many BusStop objects

Road
  contains one Lane per travel direction

Lane
  contains Vehicle objects
  has a back-reference to its Road
```

`RoadNetwork` is the world. `Road` is one line segment. **`Lane` is the important traffic container** - every vehicle belongs to exactly one lane, so it has a fixed direction and road.

grid.txt format

Small domain-specific format. Separates map data from Java code.

```
ROAD 55 170 1045 170 50 EAST,WEST
INTERSECTION 250 170 50 30 3
BUS_STOP 430 341 EAST Central_Station
```

Line type	Fields	Meaning
ROAD	x1 y1 x2 y2 speedLimit directions	Straight road + one Lane per listed direction.
INTERSECTION	x y greenTicks yellowTicks redTicks	Signalised crossing at that coordinate.
BUS_STOP	x y direction name	Named stop on one exact lane.

Current layout: three east-west roads, three north-south roads, nine signalised crossings, no roundabouts.

NetworkLoader - the Factory

`loadFromFile` reads each non-comment line, splits on whitespace, and uses a switch expression to build the right object. Invalid counts / numbers / directions throw `InvalidNetworkException` rather than silently building a broken map.

```
String[] parts = line.split("\\s+");
switch (parts[0]) {
    case "ROAD"          -> ...
    case "INTERSECTION" -> ...
    case "BUS_STOP"      -> ...
    default -> throw new InvalidNetworkException(...);
}
```

After reading, `wireIntersections` connects an intersection to every road whose centre line passes through its coordinates. Sensors use this connection later to ignore unrelated intersections.

Australian left-side traffic

Screen coordinates: x increases rightwards, y increases downwards. A lane is shifted from the road centre by `Lane.LANE_HALF_WIDTH` (9 units).

Direction	Movement vector	Left-side lane offset	On screen
EAST	(1, 0)	(0, -9)	Upper lane of a horizontal road
WEST	(-1, 0)	(0, +9)	Lower lane of a horizontal road
SOUTH	(0, 1)	(+9, 0)	Right lane of a vertical road
NORTH	(0, -1)	(-9, 0)	Left lane of a vertical road

```
laneCentreX = roadCentreX + direction.leftX() * Lane.LANE_HALF_WIDTHH;  
laneCentreY = roadCentreY + direction.leftY() * Lane.LANE_HALF_WIDTHH;
```

Same formula is used for spawning and validating bus-stop positions. The University stop is southbound at `x=559` because the vertical road centre is `x=550` and the southbound left-side lane is 9 units to the right.

BusStop.serves(Lane)

A stop serves a lane only when **all three** conditions are true: 1. Directions match. 2. The stop sits on that lane's shifted centre line. 3. The stop lies between the road endpoints.

This prevents a bus from stopping on the wrong side of a road.

Encapsulation - recent tightening

`Lane.getVehicles()` now returns `Collections.unmodifiableList(vehicles)`, and `addVehicle` uses `addIfAbsent` + throws on duplicates. Callers that need to remove vehicles must go through `Lane.removeVehicle(v)`. `Road`, `RoadNetwork`, and `BusStop` all validate constructor args with `Objects.requireNonNull` + range checks.

If a downstream caller crashes on a `getVehicles()` mutation, they need to call `removeVehicle` instead - not the other way around.

Where you touch other pairs

- **Pair A (Engine):** Their spawner and sensor traverse your `network.getRoads()` / `road.getLanes()` / `lane.getVehicles()`. Keep the read APIs stable.
- **Pair C (Vehicles):** Vehicles hold a back-ref to their `Lane` via `Lane.setRoad` / `Vehicle.attachTo`. `addVehicle` and `removeVehicle` are the only mutation entry points.
- **Pair D (Lights):** Intersections live in your `RoadNetwork` but their state machine is Pair D's. `wireIntersections` gives them their connected roads.

Your defence questions

Why is `grid.txt` separate from Java code? Map data changes more often than logic. Separating it lets us build new maps without recompiling, and lets one loader Factory handle every future map file.

Why is `Lane` the important container, not `Road`? Every vehicle has a fixed direction. That direction pins it to exactly one lane. Putting vehicles on `Lane` (not `Road`) means the sensor can iterate one direction of traffic at a time without filtering.

How do you handle a malformed `grid.txt`? `NetworkLoader` throws `InvalidNetworkException` with a message pointing at the bad line. The engine never starts with a broken world.

Why left-side offset instead of right? Australian road rule - drivers keep left. The lane offset formula (`direction.leftX() * 9`) puts eastbound traffic on the *upper* screen lane, matching how you'd read it if the screen were a map with north pointing up.

Why does `BusStop.serves` check three conditions instead of just position? Direction and endpoint bounds prevent a stop from serving the wrong lane (e.g., a WEST bus stopping at an EAST-only stop, or a stop that happens to fall on a lane extended beyond its road segment).

Patterns you're defending

- **Factory** - `NetworkLoader.loadFromFile` and `NetworkLoader.buildDefault()`
- **Encapsulation** - unmodifiable views, `requireNonNull` guards on constructors and setters
- **Exceptions** - `InvalidNetworkException` extends `SimulationException` for clear error boundaries

What to skip in the main PDF

- Section 6 (SimulationEngine tick order) - Pair A owns the loop; you just supply the network it walks
- Section 7 (Vehicle movement math) - Pair C's
- Section 8 (Sensor logic) - Pair A's
- Section 9 (Traffic light state machine and adaptive extension) - Pair D's